

LOADING THE NECESSARY LIBRARIES

In [1]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

import nltk
from nltk.corpus import stopwords
from nltk.util import ngrams
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk import pos_tag
from nltk.corpus import wordnet

import re

import string
from string import punctuation

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn import metrics
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.preprocessing import LabelBinarizer

from bs4 import BeautifulSoup

import unicodedata
```

IMPORTING THE ISOT DATASET

```
In [2]: real = pd.read_csv("/Users/moe/Desktop/ISOT_News_Dataset/True.csv")  
fake = pd.read_csv("/Users/moe/Desktop/ISOT_News_Dataset/Fake.csv")
```

```
In [3]: real['Label'] = 1  
fake['Label'] = 0
```

```
In [4]: df = pd.concat([real, fake])
```

```
In [5]: df['text'] = df['text'] + " " + df['title']  
del df['title']  
del df['subject']  
del df['date']
```

CLEANING THE DATA

In [6]:

```
stop = set(stopwords.words('english'))
punctuation = list(string.punctuation)
stop.update(punctuation)

def strip_html(text):
    soup = BeautifulSoup(text, "html.parser")
    return soup.get_text()

def remove_between_square_brackets(text):
    return re.sub(r'\[[^\]]*\]', '', text)

def remove_urls(text):
    return re.sub(r'http\S+', '', text)

def remove_stopwords(text):
    final_text = []
    for i in text.split():
        if i.strip().lower() not in stop:
            final_text.append(i.strip())
    return " ".join(final_text)

def clean_text(text):
    text = re.sub(r'[\.\*\?\]\]', '', text)
    text = re.sub(r'https?://\S+|www\.\S+', '', text)
    text = re.sub(r'<.*?>+', '', text)
    text = re.sub(r'[%s]' % re.escape(string.punctuation), '', text)
    text = re.sub(r'\n', '', text)
    text = re.sub(r'\w*\d\w*', '', text)
    text = str(text).lower()
    return text

def denoise_text(text):
    text = strip_html(text)
    text = remove_between_square_brackets(text)
    text = remove_urls(text)
    text = remove_stopwords(text)
    text = clean_text(text)
    return text

df['text'] = df['text'].apply(denoise_text)
```

Lemmatize words

```
In [7]: def lemmatize_words(text):  
        wnl = nltk.stem.WordNetLemmatizer()  
        lem = ' '.join([wnl.lemmatize(word) for word in text.split()])  
        return lem  
  
df['text'] = df['text'].apply(lemmatize_words)
```

Splitting the data into 2 parts - training and testing data

```
In [8]: x_train,x_test,y_train,y_test = train_test_split(df.text,df.Label,random_state = 0)
```

Using Bag of words model for vectorization

```
In [9]: count_vectorizer = CountVectorizer(stop_words='english')  
x_train = count_vectorizer.fit_transform(x_train.values)  
x_test = count_vectorizer.transform(x_test.values)
```

MODEL

Using 2 different models with 2 different parameters for parameter investigation values of alpha and c

Naive Bayes

In [10]:

```
#Model 1 - default parameters
from sklearn.metrics import classification_report

nb_classifier1 = MultinomialNB()
nb_classifier1.fit(x_train, y_train)

pred1 = nb_classifier1.predict(x_test)

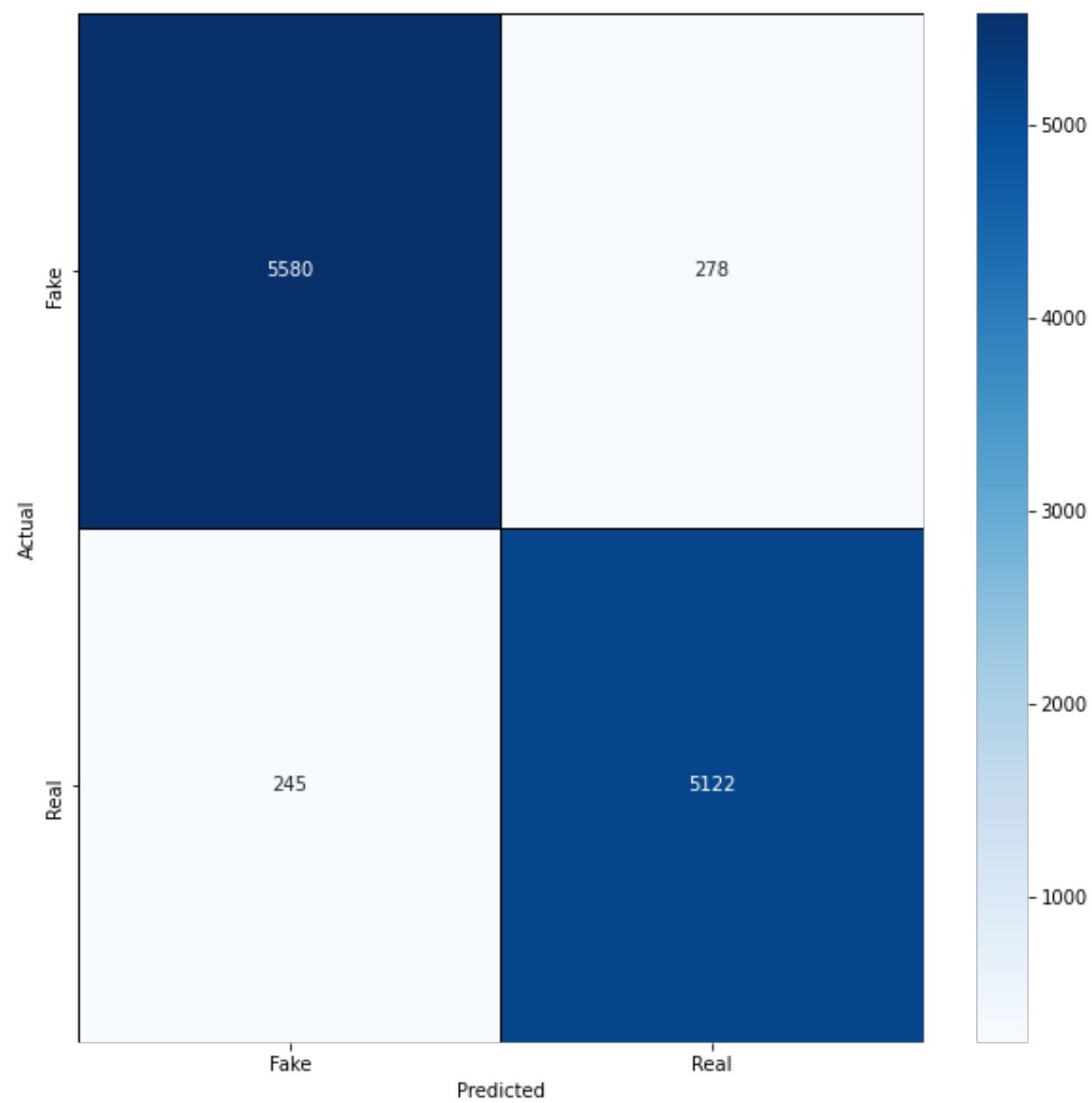
print(classification_report(y_test, pred1, target_names = ['Fake', 'Real']))
```

	precision	recall	f1-score	support
Fake	0.96	0.95	0.96	5858
Real	0.95	0.95	0.95	5367
accuracy			0.95	11225
macro avg	0.95	0.95	0.95	11225
weighted avg	0.95	0.95	0.95	11225

In [11]:

```
cml = confusion_matrix(y_test, pred1)
cml = pd.DataFrame(cml, index = ['Fake', 'Real'], columns = ['Fake', 'Real'])
plt.figure(figsize = (10,10))
sns.heatmap(cml, cmap= "Blues", linecolor = 'black', linewidth = 1, annot = True, fmt='', xticklabels = ['Fake',
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

Out[11]: Text(69.0, 0.5, 'Actual')



In [12]:

```
#Model 2
nb_classifier2 = MultinomialNB(alpha = 1000)
nb_classifier2.fit(x_train, y_train)

pred2 = nb_classifier2.predict(x_test)

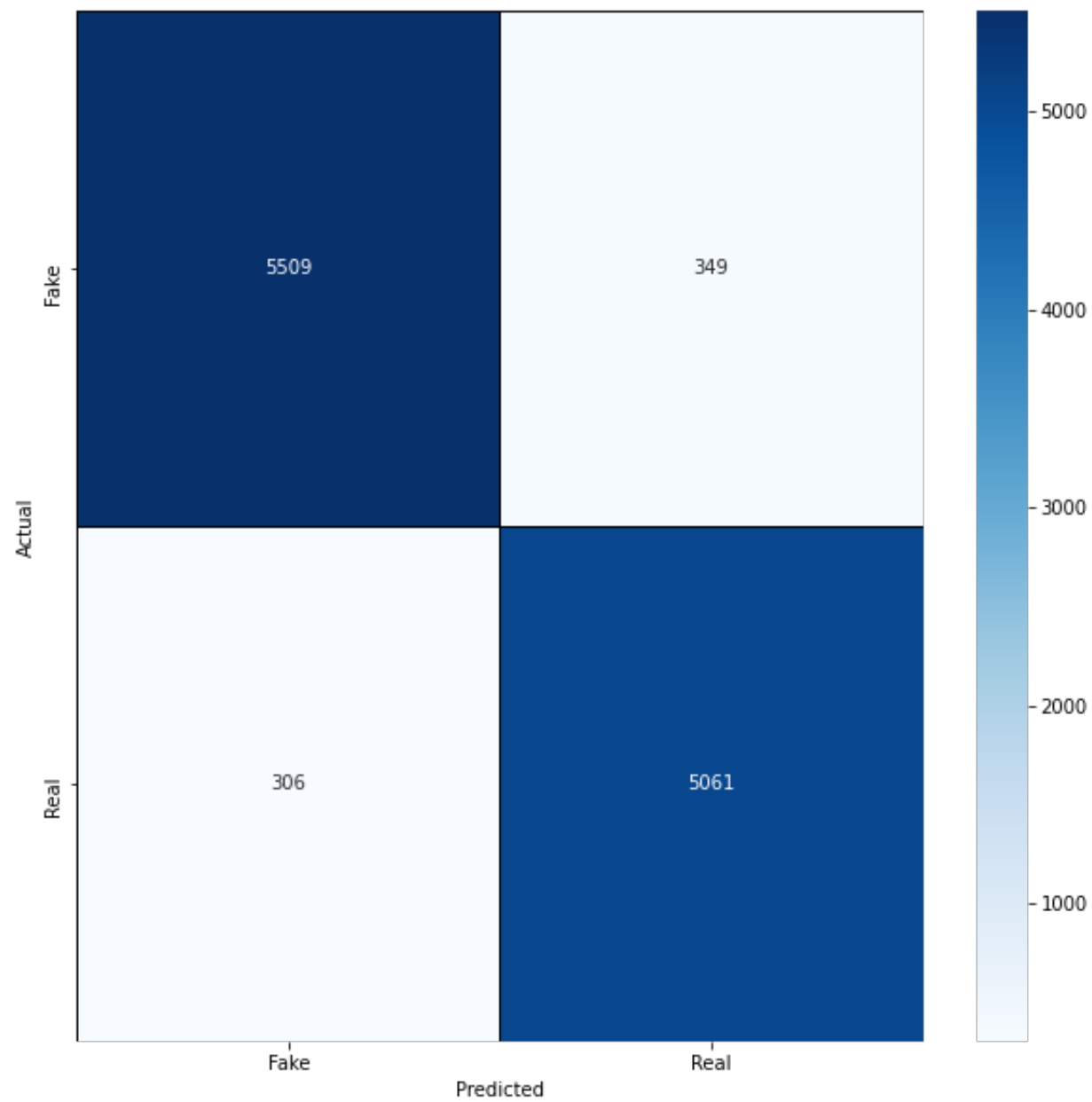
print(classification_report(y_test, pred2, target_names = ['Fake', 'Real']))
```

	precision	recall	f1-score	support
Fake	0.95	0.94	0.94	5858
Real	0.94	0.94	0.94	5367
accuracy			0.94	11225
macro avg	0.94	0.94	0.94	11225
weighted avg	0.94	0.94	0.94	11225

In [13]:

```
cm2 = confusion_matrix(y_test, pred2)
cm2 = pd.DataFrame(cm2, index = ['Fake', 'Real'], columns = ['Fake', 'Real'])
plt.figure(figsize = (10,10))
sns.heatmap(cm2, cmap= "Blues", linecolor = 'black', linewidth = 1, annot = True, fmt='', xticklabels = ['Fake',
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

Out[13]: Text(69.0, 0.5, 'Actual')



Support Vector Machine (SVM)

In [14]:

```
#Model 1
from sklearn.svm import SVC

svc_model1 = SVC(C=1, kernel='linear', gamma= 1)
svc_model1.fit(x_train, y_train)

prediction1 = svc_model1.predict(x_test)

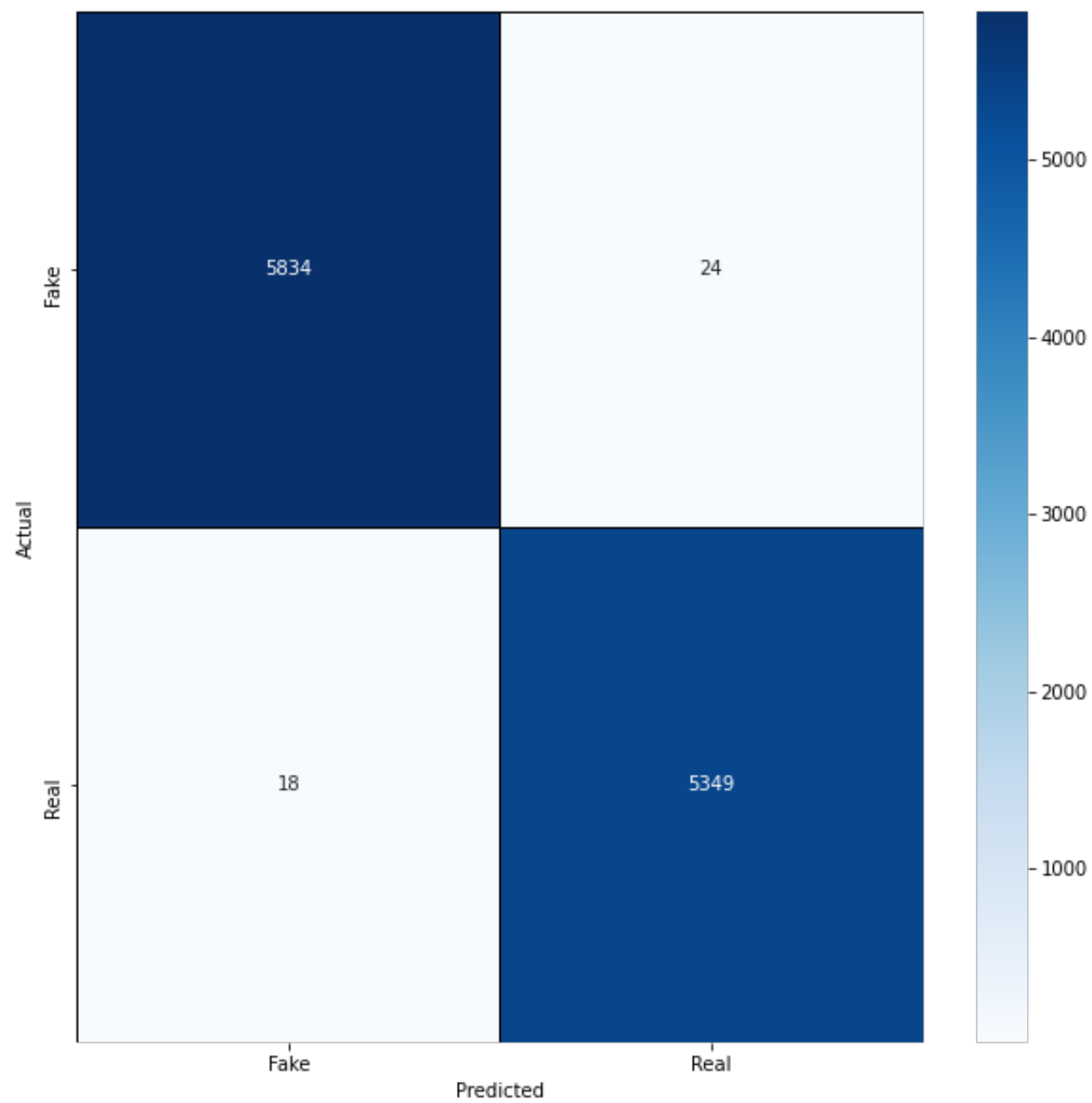
print(classification_report(y_test, prediction1, target_names = ['Fake', 'Real']))
```

	precision	recall	f1-score	support
Fake	1.00	1.00	1.00	5858
Real	1.00	1.00	1.00	5367
accuracy			1.00	11225
macro avg	1.00	1.00	1.00	11225
weighted avg	1.00	1.00	1.00	11225

In [15]:

```
cm3 = confusion_matrix(y_test,prediction1)
cm3 = pd.DataFrame(cm3 , index = ['Fake', 'Real'] , columns = ['Fake', 'Real'])
plt.figure(figsize = (10,10))
sns.heatmap(cm3,cmap= "Blues", linecolor = 'black' , linewidth = 1 , annot = True, fmt='', xticklabels = ['Fake',
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

Out[15]: Text(69.0, 0.5, 'Actual')



In [16]:

```
#Model 2
svc_model2 = SVC(C= 100, kernel='linear', gamma= 1)
svc_model2.fit(x_train, y_train)

prediction2 = svc_model2.predict(x_test)

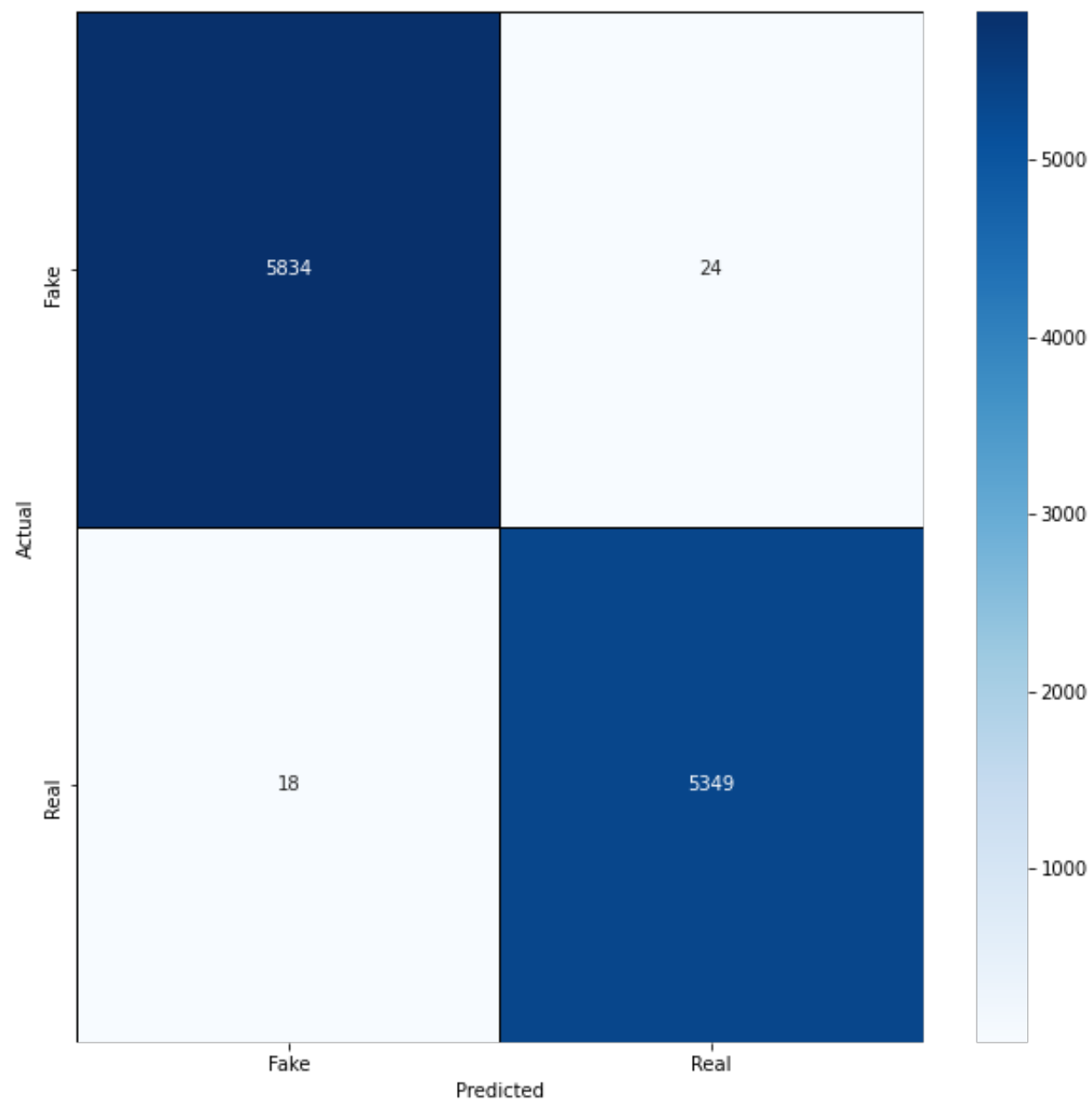
print(classification_report(y_test, prediction2, target_names = ['Fake', 'Real']))
```

	precision	recall	f1-score	support
Fake	1.00	1.00	1.00	5858
Real	1.00	1.00	1.00	5367
accuracy			1.00	11225
macro avg	1.00	1.00	1.00	11225
weighted avg	1.00	1.00	1.00	11225

In [17]:

```
cm4 = confusion_matrix(y_test, prediction2)
cm4 = pd.DataFrame(cm4, index = ['Fake', 'Real'], columns = ['Fake', 'Real'])
plt.figure(figsize = (10,10))
sns.heatmap(cm4, cmap= "Blues", linecolor = 'black', linewidth = 1, annot = True, fmt='', xticklabels = ['Fake',
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

Out[17]: Text(69.0, 0.5, 'Actual')



In []: